

Java Multithreading & Concurrency

A-Z Interview Revision Notes

Simple English - Real Life Examples - ASCII Diagrams - Interview Questions - Revision Sheet

Use this as your fast study guide before moving to Java 8, JVM, Spring Boot, and System Design.

What this PDF covers

- Process vs Thread
- Multithreading, Concurrency, Parallelism
- Thread life cycle
- How to create threads in Java
- start() vs run()
- sleep(), join(), yield()
- Race condition and thread safety
- synchronized, locks, volatile, atomic classes
- ExecutorService, Callable, Future, CompletableFuture
- Interview questions and final revision map

1. Process and Thread - the foundation

Process = a running program. Thread = a worker inside that running program.

```
Process
|
+--> Thread 1
+--> Thread 2
+--> Thread 3
```

Real life: one restaurant is the process. Cook, waiter, cashier, and cleaner are threads.

```
Restaurant (Process)
|- Cook
|- Waiter
|- Cashier
|- Cleaner
```

- Processes have separate memory.
- Threads in the same process share memory.
- Threads are lighter and faster to create than processes.

2. Multithreading, Concurrency, Parallelism

- Multithreading = multiple threads inside one process.
- Concurrency = handling multiple tasks by taking turns.
- Parallelism = multiple tasks running at the same time on multiple cores.

```
Concurrency (one worker)
Task A -> Task B -> Task C -> Task A
```

Parallelism (many workers)

Core 1 -> Task A

Core 2 -> Task B

Core 3 -> Task C

- Single-core CPU: concurrency, not true parallelism.
- Multi-core CPU: true parallelism is possible.

3. Thread life cycle

- NEW - thread object created, not started.
- RUNNABLE - ready to run; waiting for CPU or currently executing.
- BLOCKED - waiting for a lock.
- WAITING - waiting indefinitely for another thread/event.
- TIMED_WAITING - waiting for a fixed time.
- TERMINATED - thread finished execution.

```
NEW -> start() -> RUNNABLE -> CPU executes -> TERMINATED
                        |
                        +--> BLOCKED / WAITING / TIMED_WAITING
```

4. Creating threads in Java

- Way 1: extend Thread and override run().
- Way 2: implement Runnable and pass it to Thread.
- Runnable is usually preferred because it separates the task from the worker and supports single inheritance.

```
class MyTask implements Runnable {
    public void run() {
        System.out.println("Task running");
    }
}
```

```
Thread t = new Thread(new MyTask());
t.start();
```

5. start() vs run()

- start() creates a new thread and then JVM calls run() on that thread.
- run() directly is just a normal method call on the current thread.

```
t.start();    // new thread
t.run();      // no new thread, normal method call
```

- Interview answer: start() = real multithreading. run() = ordinary method call.

6. Scheduling, sleep(), join(), yield()

- The OS scheduler decides which runnable thread gets CPU time.
- sleep(time) pauses the current thread for a given time.

- `join()` makes one thread wait until another thread finishes.
- `yield()` politely suggests that the scheduler may run another thread.

```
Thread.sleep(3000);    // current thread sleeps
otherThread.join();    // wait for other thread
Thread.yield();        // optional hint to scheduler
```

7. Race condition and thread safety

- Race condition happens when two or more threads access shared data and the result depends on timing.
- Thread safety means the code behaves correctly even when many threads use it at the same time.

```
int balance = 1000;

// Thread 1: balance = balance + 500;
// Thread 2: balance = balance - 300;
// If timing overlaps, final value may be wrong
```

- Shared mutable state is the main danger in multithreading.

8. synchronized, locks, deadlock, starvation

- `synchronized` ensures only one thread at a time enters a critical section.
- Object lock = used for synchronized instance methods or blocks.
- Class lock = used for static synchronized methods or blocks.
- Deadlock = two or more threads wait for each other forever.
- Starvation = one thread keeps waiting because others keep getting priority.

```
synchronized (this) {
    // critical section
}
public synchronized void update() {
    // one thread at a time
}
```

9. volatile and atomic classes

- `volatile` makes a variable visible across threads faster, but it does not make compound operations atomic.
- `AtomicInteger` and similar classes provide lock-free atomic operations for common counters and flags.

```
volatile boolean running = true;
AtomicInteger count = new AtomicInteger(0);
count.incrementAndGet();
```

10. ExecutorService and thread pools

- Do not create raw threads everywhere in real applications.
- Use `ExecutorService` to reuse threads and manage task execution cleanly.

- Thread pool = a fixed set of threads that repeatedly take new tasks.

```
ExecutorService pool = Executors.newFixedThreadPool(4);
pool.submit(() -> System.out.println("task"));
pool.shutdown();
```

11. Callable, Future, CompletableFuture

- Runnable cannot return a value. Callable can return a value and throw checked exceptions.
- Future represents the result of an asynchronous task.
- CompletableFuture is a modern API for async pipelines and chaining tasks.

```
Callable<Integer> task = () -> 10;
Future<Integer> f = pool.submit(task);
Integer value = f.get();
CompletableFuture.supplyAsync(() -> fetchData())
    .thenApply(data -> transform(data))
    .thenAccept(System.out::println);
```

12. Fast interview answers

- Process = running program. Thread = smallest unit of execution inside a process.
- start() creates a new thread; run() does not.
- Concurrency = taking turns; Parallelism = running together.
- sleep() pauses current thread; join() waits for another thread; yield() is only a hint.
- Runnable is preferred over extending Thread.
- synchronized protects critical sections from race conditions.

13. What to learn after this

- Java 8 Features - lambda, stream API, Optional, method references.
- JVM Internals - memory areas, garbage collection, class loading.
- Design Patterns - Singleton, Factory, Strategy, Observer, Builder, Adapter.
- Spring Boot - REST API, dependency injection, autoconfiguration, profiles.
- System Design - caching, load balancing, database scaling, queues, rate limiting.

14. One-page revision map

Java Thread Journey

1. NEW

Thread object created

2. RUNNABLE

start() called, waiting for CPU

3. CPU EXECUTING (RUNNABLE)

CPU is executing run()

4. BLOCKED / WAITING / TIMED_WAITING
waiting for lock, thread, or time

5. TERMINATED
run() finished

Key idea: shared memory makes threads powerful, but also dangerous if data is modified without protection.

15. Final self-check quiz

1. What is the difference between process and thread?
2. Why is concurrency not the same as parallelism?
3. What happens when you call run() directly?
4. Why is sleep() not the same as join()?
5. What problem does synchronized solve?
6. Why can shared variables become dangerous in multithreading?

End goal: you should be able to explain every thread topic in plain English, draw a simple diagram, and answer the common interview questions without memorizing long theory.